

MODULE / 07

Memory, Context & State.

How agents remember — across turns, sessions, and lifetimes.

short-term · long-term · vector · structured · persistence · context window

Core concepts.

The mental model — pick the right memory for the right job.

01

Short-term ↔ Long-term

Short-term — the live conversation buffer; volatile, fast, scoped to one session. Lost on restart.

Long-term — persisted across sessions in a store (DB, file, vector index). Survives restarts, grows over time, must be retrieved deliberately.

02

Vector ↔ Structured

Vector — embeddings + similarity search. Great for fuzzy recall: "what did we discuss about pricing?"

Structured — SQL / KV / JSON schema. Great for exact facts: user_id, preferences, order history, profile fields.

03

Conversation persistence

Save every turn keyed by `session_id`. Replay on reconnect so the agent picks up exactly where it left off. Patterns: SQLite/Redis for chat logs, summaries for compression, TTL for privacy.

04

Context window optimization

Tokens are finite & expensive. Use **summarization** for old turns, **sliding windows** for recent, and **retrieval** (RAG) to inject only what's relevant — never the full history.

Build it.

Two labs — a persistent memory agent and a session-aware assistant.

LAB 01

Persistent memory agent

Agent that remembers user facts across runs using a vector store + structured profile table.

```
# pip install chromadb openai sqlite-utils
from chromadb import PersistentClient
import sqlite3, openai

mem = PersistentClient("./mem").get_or_create_collection("facts")
db = sqlite3.connect("profile.db")

def remember(user, text):
    mem.add(documents=[text], ids=[f"{user}-{uuid4()}"])

def recall(user, query, k=5):
    return mem.query(query_texts=[query], n_results=k)
```

Goal → Run the script twice. On the 2nd run the agent should greet the user by name and recall facts from the 1st run.

LAB 02

Session-aware assistant

Multi-session chat — each `session_id` has its own running history, summary, and context budget.

- 01 → Persist messages in SQLite keyed by `session_id`.
- 02 → On each turn, load last N=10 messages.
- 03 → When history > 4k tokens, summarize the oldest half & replace.
- 04 → Retrieve top-3 long-term memories via vector search.
- 05 → Assemble prompt: *summary + recent + retrieved + user*.
- 06 → Stream reply, persist, repeat.

Stretch → Add 2 concurrent sessions and verify their contexts never bleed into each other.